ST303 - Stochastic Simulation 2024/25 Group Project

Group 6

Candidate numbers:

40073

36455

45997

42995

36999

Question 1

a)

The true values

We compute the exact values of C(t) for t values from 1 to 100.

Monte Carlo Methods

We take n=10^6.

Transformation 1

For transformation 1, we use $u = sinx$, then $x = arcsin(u)$. When $x = 0$, then $u = 0$, and when $x = \pi/2$, then $u = 1$. We compute: Using for loop to investigate the performance of integral across different values of t:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t \, dx \overset{x=\arcsin(u)}{=} \int_0^1 u^t [\cos(\arcsin(u)]^{-1} + [\cos(\arcsin(u)]^{t-1} \, du \overset{U \overset{d}{=} U(0,1)}{=}$$

$$\mathbb{E}\left[u^t [\cos(\arcsin(u)]^{-1} + [\cos(\arcsin(u)]^{t-1}\right] \overset{MC}{\approx} \sum_{i=1}^{N} [u^t [\cos(\arcsin(u)]^{-1} + [\cos(\arcsin(u)]^{t-1}]$$

Transformation 2

For transformation 2, we use $u = cosx$, then $x = \arccos(u)$. When $x = 0$, then $u = 1$, and when $x = \pi/2$, then $u = 0$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t \, dx \overset{x=\arccos(u)}{=} \int_0^1 u^t [\sin(\arccos(u)]^{-1} + [\sin(\arccos(u)]^{t-1} \, du \overset{U \overset{d}{=} U(0,1)}{=}$$

$$\mathbb{E}\left[u^t [\sin(\arccos(u)]^{-1} + [\sin(\arccos(u)]^{t-1}\right] \overset{MC}{\approx} \sum_{i=1}^{N} [u^t [\sin(\arccos(u)]^{-1} + [\sin(\arccos(u)]^{t-1}]$$

Using for loop to investigate the performance of integral across different values of t:

According to the above results, transformation 1 and 2 have exactly the same mean and standard error.

Transformation 3

For transformation 3, we use $u = \frac{2}{\pi}x$, then $x = \frac{\pi}{2}u$. When $x = 0$, then $u = 0$, and when $x = \pi/2$, then $u = 1$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t \, dx \overset{x=\frac{\pi}{2}u}{=} \int_0^1 [(\sin((\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t] \frac{\pi}{2} \, du \overset{U \overset{d}{=} U(0,1)}{=}$$

$$\mathbb{E}\left[\frac{\pi}{2}[(\sin((\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t]\right] \overset{MC}{\approx} \sum_{i=1}^{N} \frac{\pi}{2}[(\sin((\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t]$$

Using for loop to investigate the performance of integral across different values of t:

The following table shows the comparison between different methods:

```
##     Transformation     Mean_Diff        Mean_Standard_Error
## 1 Transformation 1   -3.703208e-04       0.0026780224
## 2 Transformation 2   -3.703208e-04       0.0026780224
## 3 Transformation 3   -1.242511e-05       0.0004949451
```

According to the transformation methods above, transformation 3 has the smallest difference between the estimated mean and actual mean, and also the smallest standard error. Thus, we do the variance reduction based on transformation 3.

The antithetic variate reduction is not applicable since the function is not monotone. Then we try the control variates method first.

1. Control variates method

We want to find a random variable Y with mean 0 and a good constant c such that $\mathbb{E}(X + cY) \approx \theta$.

Control Variate Method

Stratified Sampling

We estimate:

$$h(U) = \left[\left(\sin\left(\frac{\pi}{2}\frac{U_j + j - 1}{n}\right)\right)^t + \left(\cos\left(\frac{\pi}{2}\frac{U_j + j - 1}{n}\right)\right)^t\right]\frac{\pi}{2}$$

Stratified Sampling + Control Variates

Compare all variance reduction methods

This table represents the differences between two variance reduction methods (SS+CV, only CV) with Monte Carlo Methods for $t = 1,2,\ldots,10$.

```
compare_df <- data.frame(
  t = 1:10,
  diff_SSCV = (abs(Diff_3) - abs(Diff_cvss))[1:10],
  sd_improved_SSCV = (round((Std_Error_3 - SE_cvss)/Std_Error_3, digits = 2))[1:10],
  diff_CV = (abs(Diff_3) - abs(Diff_cv3))[1:10],
  sd_improved_CV = (round((Std_Error_3 - SE_cv3)/Std_Error_3, digits = 2))[1:10]
)
compare_df
```

```
##      t    diff_SSCV     sd_improved_SSCV   diff_CV   sd_improved_CV
## 1    1 6.811730e-05            0.60                     0          0.60
## 2    2 0.000000e+00            0.00                     0          0.00
## 3    3 1.591679e-04            0.49                     0          0.49
## 4    4 2.628563e-05            0.63                     0          0.63
## 5    5 8.088983e-06            0.65                     0          0.65
## 6    6 1.334485e-04            0.65                     0          0.65
## 7    7 3.564456e-04            0.64                     0          0.64
## 8    8 1.293248e-04            0.62                     0          0.62
## 9    9 3.460293e-04            0.60                     0          0.60
## 10 10 7.889249e-07             0.58                     0          0.58
```

(b)

Calculate the true value of the double integral

```
set.seed(1)
f <- function(x, y) {
  r <- sqrt(x^2 + y^2)
  ifelse(r == 0, 1, sin(r) / r)  # Handle the removable singularity at (0,0)
}

result <- integrate(function(y) {
  sapply(y, function(yi) {
    integrate(function(x) f(x, yi), -1, 1)$value
  })
}, -1, 1)

# Print the result
true_integral_1b <- result$value
true_integral_1b
```

```
## [1] 3.575761
```

Pure Monte Carlo Methods

Since the equation is symmetric, we can compute:

$$\int_{-1}^{1}\int_{-1}^{1}\frac{\sin(\sqrt{x^2+y^2})}{\sqrt{x^2+y^2}}\,dx\,dy = 4\int_{0}^{1}\int_{0}^{1}\frac{\sin(\sqrt{x^2+y^2})}{\sqrt{x^2+y^2}}\,dx\,dy \overset{(X,Y)\overset{ind}{\sim}\overset{d}{\sim}\mathcal{U}(0,1)}{=}$$
$$4\mathbb{E}\left[\frac{\sin(\sqrt{x^2+y^2})}{\sqrt{x^2+y^2}}\right] \overset{MC}{\approx} \frac{4}{N}\sum_{i=1}^{N}\frac{\sin(\sqrt{x^2+y^2})}{\sqrt{x^2+y^2}}\,dx\,dy,$$

where $X_1,\dots,X_N \overset{d}{\sim} \mathcal{U}(0,1)$, $Y_1,\dots,Y_N \overset{d}{\sim} \mathcal{U}(0,1)$, and the samples are independent.

1. Antithetic Variate Reduction

We need to calculate

$$\frac{4}{N}\sum_{i=1}^{N}\frac{\sin(\sqrt{(x^2+y^2)})}{\sqrt{x^2+y^2}} = \frac{4}{N}\sum_{i=1}^{N}\frac{\sin(\sqrt{((1-x)^2+(1-y)^2)})}{\sqrt{(1-x)^2+(1-y)^2}},$$

where $X_1,\dots,X_N \overset{d}{\sim} \mathcal{U}(0,1)$, $Y_1,\dots,Y_N \overset{d}{\sim} \mathcal{U}(0,1)$, and the samples are independent.

```
improve_av <- (Std_Error_mc - Std_Error_av)/Std_Error_mc
improve_av
```

```
## [1] 0.7587932
```

The antithetic variate reduction reduces the standard error by 75.8793%.

2. Stratified Sampling

Since $1-U_i \overset{d}{=} U_i$, we can compute

$$h(U) = \frac{4}{N}\sum_{i=1}^{N}\frac{\sin\left(\sqrt{\left(\frac{U_i+i-1}{n}\right)^2+\left(\frac{U_j+j-1}{n}\right)^2}\right)}{\sqrt{\left(\frac{U_i+i-1}{n}\right)^2+\left(\frac{U_j+j-1}{n}\right)^2}},$$

where $X_1,\dots,X_N \overset{d}{\sim} \mathcal{U}(0,1)$, $Y_1,\dots,Y_N \overset{d}{\sim} \mathcal{U}(0,1)$, and the samples are independent.

```
improve_SS <- (EstIntegral_mc - Mu_SS)/EstIntegral_mc
improve_SS
```

```
## [1] -0.001564384
```

Stratified Sampling + Antithetic Variate reduction

To combine the stratified sampling with antithetic variate reduction, we calculate the mean of the following to estimations:

$$h(U) = \frac{4}{N}\sum_{i=1}^{N}\frac{\sin(\sqrt{\left(\frac{U_i+i-1}{n}\right)^2+\left(\frac{U_j+j-1}{n}\right)^2})}{\sqrt{\left(\frac{U_i+i-1}{n}\right)^2+\left(\frac{U_j+j-1}{n}\right)^2}},$$

and

$$h(1-U) = \frac{4}{N}\sum_{i=1}^{N}\frac{\sin(\sqrt{\left(\frac{i-U_i}{n}\right)^2+\left(\frac{j-U_j}{n}\right)^2})}{\sqrt{\left(\frac{i-U_i}{n}\right)^2+\left(\frac{j-U_j}{n}\right)^2}}.$$

```
improve_SS <- (EstIntegral_mc - Mu_avss)/EstIntegral_mc
improve_SS

## [1] -0.001564385
```

The following table shows the comparison between different methods:

```
##          Method         Mean             Diff                  Standard_error
## 1 Ture Value_1b     3.575761               NA                              NA
## 2         MC_1b     3.575741  2.05941642512641e-05     0.000821174490111744
## 3         AV_1b     3.575570   0.000191293603578391     0.000198072897562496
## 4         SS_1b     3.581334  -0.00557323728978476     0.00115390418221002
## 5       AVSS_1b     3.581335  -0.00557323939834831     0.00115390416790356
```

Therefore, the antithetic variate reduction can reduce the variance by 75.8793%, and the difference between the estimated mean and the true mean is the smallest. Therefore, the antithetic variate reduction is the best method to improve accuracy.

Question 2

(a)

We generate 10 values for each of the parameters m, ν, λ, α, based on the conditions on their values specified in the question. They are shuffled so that $10^4 = 10000$ distinct sets of parameters are created.

```
m_list <- seq(1.6, 10, length.out = 10)
nu_list <- seq(-5, 5, length.out = 10)
lambda_list <- seq(-5, 5, length.out = 10)
alpha_list <- seq(0.1, 10, length.out = 10)

groups <- expand.grid(m_list, nu_list, lambda_list, alpha_list)

m <- groups$Var1
nu <- groups$Var2
lambda <- groups$Var3
alpha <- groups$Var4
```

We also calculate the theoretical mean and variance for each set of parameters using the formula from part (b):

```
theoretical_mean <- lambda - alpha*nu/(2*(m - 1))
theoretical_var <- alpha^2/(2*m - 3) * (1 + nu^2/(4*(m - 1)^2))
```

Next, we create a dataframe summarising all information about the future simulated datasets, which is called `Q2_summary`. This dataframe has 10000 rows, each row representing the information of one set of parameters generated before.
Later, we will run R codes across all sets of parameters to get the **sample mean**, **sample variance** and **empirical acceptance rate** for each envelope function and each set of parameters. These numbers will be integrated into `Q2_summary` dataframe as new columns so that we can compare the performance of different envelope functions later.

```
Q2_summary <- data.frame(m, nu, lambda, alpha,
                         theoretical_mean, theoretical_var)
head(Q2_summary)

##          m nu lambda alpha theoretical_mean theoretical_var
## 1 1.600000 -5     -5   0.1       -4.583333     0.918055556
## 2 2.533333 -5     -5   0.1       -4.836957     0.017701537
## 3 3.466667 -5     -5   0.1       -4.898649     0.005153923
## 4 4.400000 -5     -5   0.1       -4.926471     0.002656306
## 5 5.333333 -5     -5   0.1       -4.942308     0.001738487
## 6 6.266667 -5     -5   0.1       -4.952532     0.001285305
```

Now we would like to deal with the target function $f(x)$. With transformation

$$y = \tan^{-1}\left(\frac{x-\lambda}{\alpha}\right), y \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$$

We get

$$f(y) \propto \big(\cos(y)\big)^{2m} e^{-\nu y}$$

and we are able to derive three envelop functions from that:
Envelope function 1 - this is essentially a uniform distribution

$$g(y) = \frac{1}{\pi}$$

Envelope function 2

$$g(y) \propto e^{-\nu y}$$

Envelope function 3

$$g(y) \propto \cos(y)$$

We can derive two more envelope functions from the original form of the target function, i.e.

$$f(x) \propto \left[1 + \left(\frac{x-\lambda}{\alpha}\right)\right]^{-m} e^{-v\, arctan\left(\frac{x-\lambda}{\alpha}\right)}$$

Envelope function 4 - after writing it down, we noticed that this is essentially a Cauchy distribution

$$g(y) \propto \left(1 + \left(\frac{y-\lambda}{\alpha}\right)^2\right)^{-1}$$

Envelope function 5

$$g(y) \propto \left(1 + \left(\frac{y-\lambda}{\alpha}\right)^2\right)^{-m}$$

By setting $m = \frac{k+1}{2}$ and $z = \frac{\sqrt{k}(y-\lambda)}{\alpha}$, we transformed it into a form similar to a t-distribution:

$$g(y) \propto \left(1 + \frac{z^2}{k}\right)^{-\frac{k+1}{2}}$$

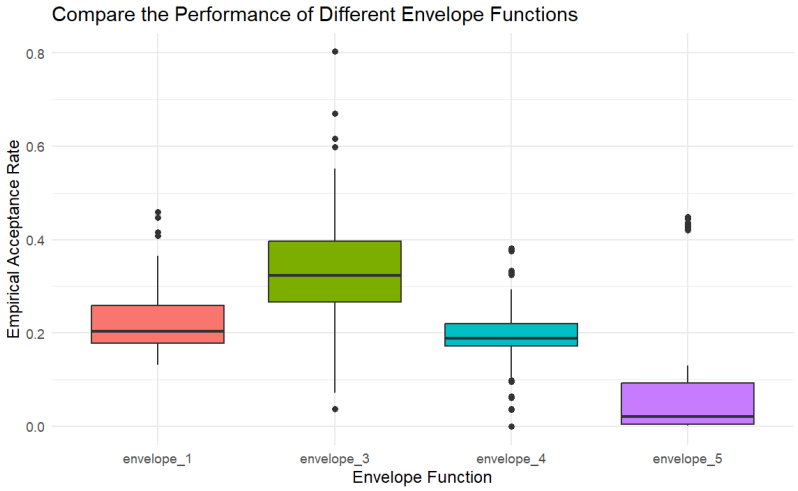
Unfortunately, due to the limit on number of pages, we are unable to explain every algorithm we wrote in R. We will elaborate on the best envelope function we picked later, after the best envelope function is picked.
R will take approximately 12 minutes to run the codes for all 5 envelope functions. That is a reasonable efficiency for 10000 sets of parameters.

Pick the best envelope function

When writing the codes, we noticed that envelope function 2 is not a practical choice, as it will collapse every time m becomes a decimal. Hence, we kicked it out from the further analysis of performances.

To pick the best envelope function, we compare **a)** the empirical rate of acceptance and **b)** the time of running the acceptance-rejection algorithm.

We summarised the empirical rate of acceptance of these envelope function in the boxplot below:



We summarised the time consumed to run different algorithms in the following table:

```
##                elapsed
## sim_envelope_1  196.09
## sim_envelope_3  190.15
## sim_envelope_4  290.75
## sim_envelope_5   82.25
```

We can see that generally, envelope 3 has significantly better acceptance rates than the others, while it has relatively good computational efficiency. Thus, pick envelope 3 as our envelope function.

Below are extra details about the algorithm we used for envelope 3:
With transformation $y = arctan\left(\frac{x-\lambda}{\alpha}\right)$, we have

$$f(y) \propto (\cos(y))^{2m} e^{-\nu y}$$
$$g(y) = \frac{1}{2}\cos(y)$$

Y is generated by using inverse transform algorithm:

$$F^{-1}(U) = sin^{-1}(2U - 1)$$

As we decided to pick it as our best envelope function, we need to perform variance reduction on it. Control variate method is used at this stage to reduce the variance of the sample $Y_{original}$ generated from $g(y)$. With $Y' = \frac{1}{2} - U$ being the zero-mean random variable, $Y = Y_{original} = cY'$ (where $c = -\frac{Cov(Y_{original}, Y')}{Var(Y')}$). This method reduced the variance by 89.88%.

Accept the sample from the envelope function when

$$U < \frac{\left(cos(y)\right)^{2m-1} e^{-\nu y}}{\left[cos\left(arctan\left(-\frac{\nu}{2m-1}\right)\right)\right] e^{-\nu\, arctan\left(-\frac{\nu}{2m-1}\right)}}$$

Below are the codes:

```
# Generate Y
set.seed(1)
n <- 10^5
U <- runif(n)

Y_original <- asin(2*U-1)
SE_original <- sd(Y_original)/sqrt(n)

# Control variate method
set.seed(1)
Y_prime <- 1/2 - U
c <- -12*cov(Y_original, Y_prime) # 1/12 is the population variance of 1 - U
Y <- Y_original + c*Y_prime
SE_cv <- sd(Y)/sqrt(n)
(SE_original - SE_cv)/SE_original # percentage of standard error improved
```

```
## [1] 0.8987566
```

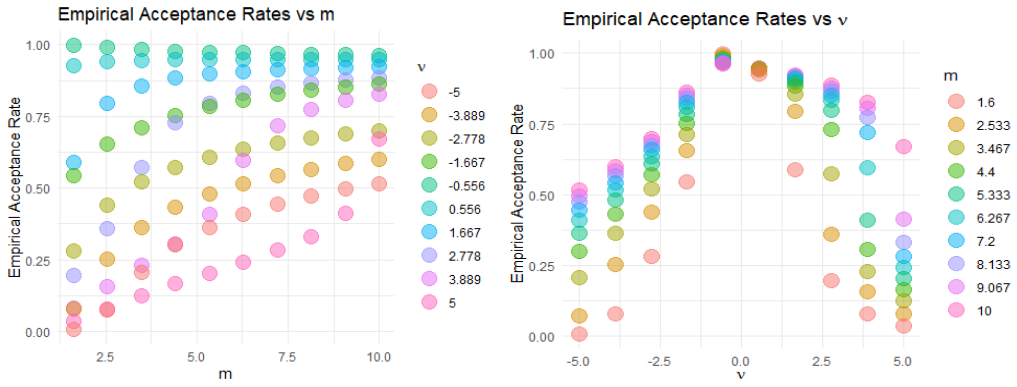
```
# Generate the sample using AR
set.seed(1)
ptm <- proc.time()
U <- runif(n)
```

```
temp <- atan(-nu/(2*m-1))

sample_envelope_3 <- lapply(1:length(nu), function(i){
  temp <- temp[i]
  m <- m[i]
  nu <- nu[i]

  Y[U < (cos(Y)^{2*m-1}*exp(-nu*Y))/(cos(temp)^{2*m-1}*exp(-nu*temp))]
})
sim_envelope_3 <- (proc.time()- ptm)[3]

# Produce X, and check mean and variance
X_envelope_3 <- lapply(1:length(nu), function(i) tan(sample_envelope_3[[i]]) * alpha[i]
 + lambda[i])

envelop_3_empirical_mean_cv <- unlist(lapply(1:length(nu), function(i) mean(X_envelope_
3[[i]])))
envelop_3_empirical_var_cv <- unlist(lapply(1:length(nu), function(i) var(X_envelope_3
[[i]])))
```

As a result of the control variates method, the empirical acceptance rate of the envelope function 3 has been improved significantly. The mean below is clearly higher than what we had in the previous boxplot.

```
# empirical rate of acceptance
envelope_3_empirical_acceptance_cv <- unlist(lapply(1:length(nu), function(i) length(sa
mple_envelope_3[[i]])/10^5))
summary(envelope_3_empirical_acceptance_cv)

##      Min.  1st Qu.   Median      Mean  3rd Qu.      Max.
## 0.00657  0.41200  0.68174  0.63328  0.89993  0.99770
```

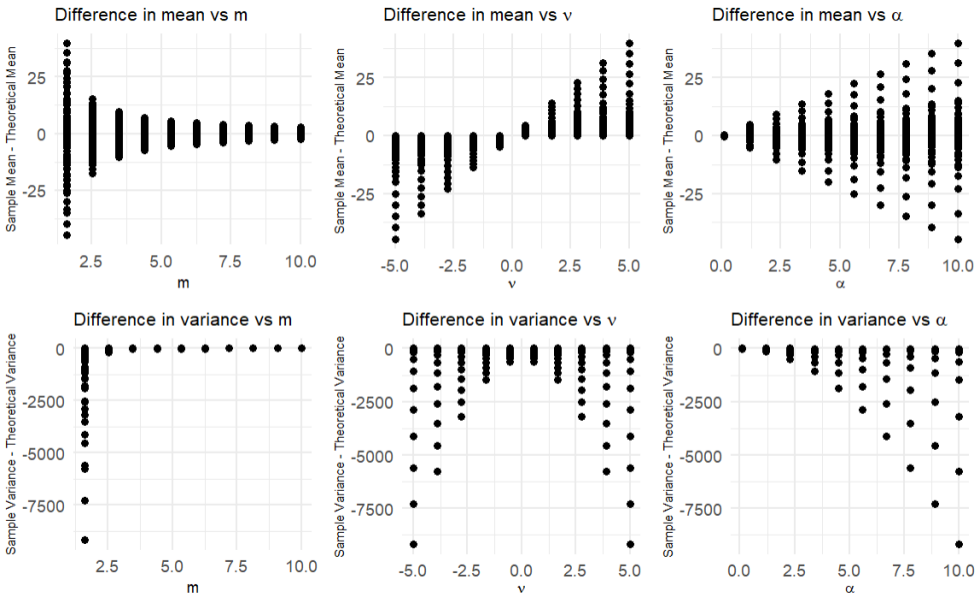
The question also asks which values of parameters will give the most efficient simulation. We will answer this question here.
By analysing the information from the updated Q2_summary dataframe with dplyr package, we are able to observe that the rate of acceptance is not affected by either λ or α. Thus, check the relationship between rate of acceptance and m, ν, using ggplot2:



From the plot, we can see that the acceptance rate is highest when m is small, **AND** ν is close to zero.

(b)

We have already calculated the theoretical mean and variances when we defined Q2_summary early on. Now, we compare them with the sample mean and variances using the dplyr package, and the differences between them are plotted. From the dataframe, we noticed that λ has no effect on either the deviation from theoretical mean or the deviation from theoretical variances. Use plots to investigate the effects of other parameters:

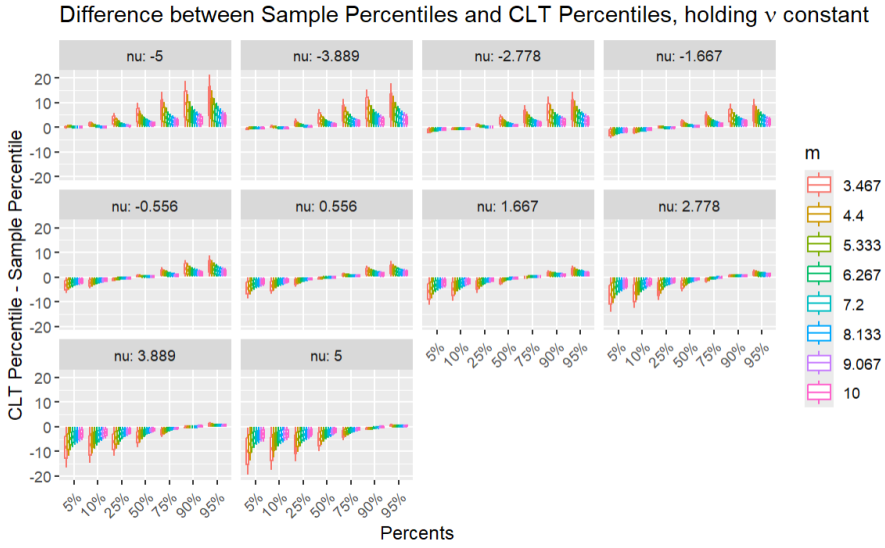


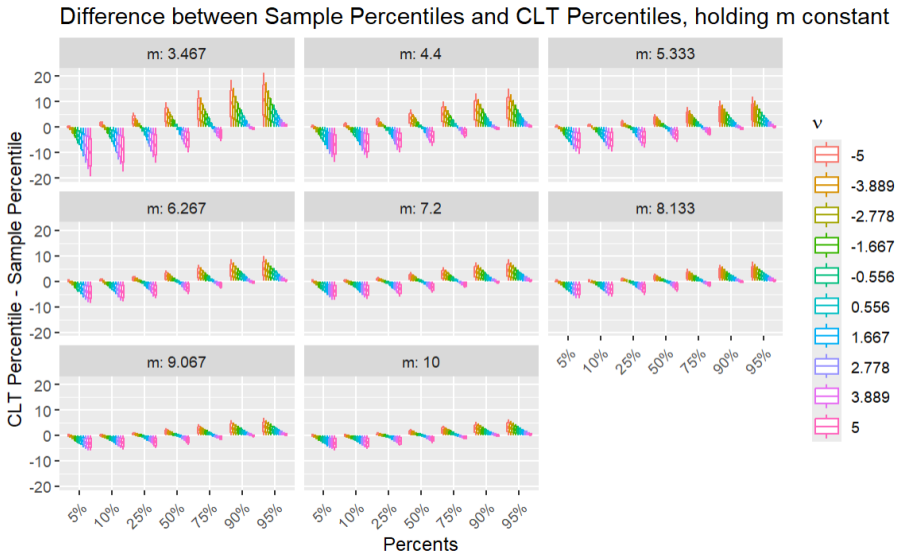
From the plots and the other analysis from the dataframe, we draw the following conclusions:
1. The differences between the sample and theoretical mean/variance is the smallest when
- m is large;
- ν is close to zero;
- α is small.

2. The mean of all differences in mean is small.
3. The mean of all differences in variance is large, and there is no empirical variance that is higher than the theoretical variance.

(c)

With the help of matrix, list and for loops, it is fairly easy to find the sample quantile for each of the 10000 sets of parameters.
For CLT quantiles, we make the unrealistic assumption of $\frac{X-E(X)}{\sqrt{Var(X)}} \sim N(0,1)$, i.e. assuming that X follows a normal distribution. The expectations and variances used here are the theoretical means and variances calculated in part (b). As a result, like the previous Q2_summary, we are able to store the difference between sample quantiles and CLT quantiles for each set of parameters in Q2_summary_quantiles_diff. We focus on 5%, 10%, 25%, 50%, 75%, 90% and 95% percentiles. We plot this information using ggplot2.
Note that we remove $m = 1.6$ and $m = 2.53$ from the plot, just to make the trend looks clearer.





From these plots one can notice the following:

1. CLT percentiles are lower than the sample percentiles when the percentage is low. CLT percentiles are higher than the sample percentiles when the percentage is high. Therefore, we can conclude that compared to a normal distribution, our sample distribution has heavier tails on both sides.

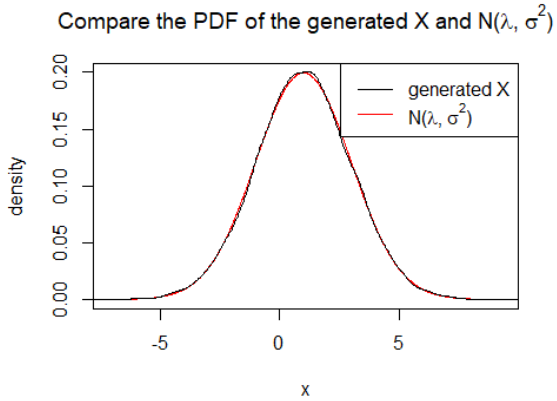
2. The higher the m, the lower the discrepancies between the CLT percentiles and the sample percentiles. Hence, as m gets higher, the sample distribution converges to a normal distribution.

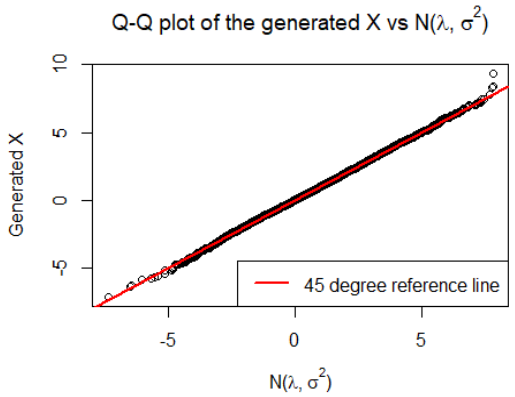
3. The closer ν is to zero, the lower discrepancies between the CLT percentiles and the sample percentiles. Hence, as ν gets closer to zero, the sample distribution converges to a normal distribution.

4. We can also see that, while m has little effect on the skewness of the sample distribution (with respect to symmetrical normal distribution) – this is indicated in the second diagram, where the general shape in subplots remained unchanged when m increases – ν, on the other hand, affects the skewness. The higher the ν, the more left-skewed the sample distribution is. This is deduced from the fact that, while the sample distribution and the normal distribution shares the same mean (which is also the median of the normal distribution), the median of the normal distribution gets smaller than the median of the sample distribution as ν increases (in the first diagram), which can only occur if the sample distribution is left-skewed.

d)

We set $m = 100$ (a large value!) and $\sigma = 2$ to test the arguments. We generate $X \sim f(x)$ with our best envelope function, perform sampling from a $N(\lambda, \sigma^2)$, and make comparison between the sample distribution and the normal distribution. With use PDF flot and Q-Q plot to draw conclusions.





Both the PDF plot and the Q-Q plot supports the argument that, under the suggested conditions, there is indeed vast similarity between the generated X and $N(\lambda, \sigma^2)$. However, one should notice that when m is large, the rate of acceptance tends to be very low (in this case 8.99%), which implies a big waste of computational power. Consequently, there may not be sufficient data to strongly support the argument.

Question 3

(a)

A Bessel process R_t of order $\delta \geq 1$ has the SDE:

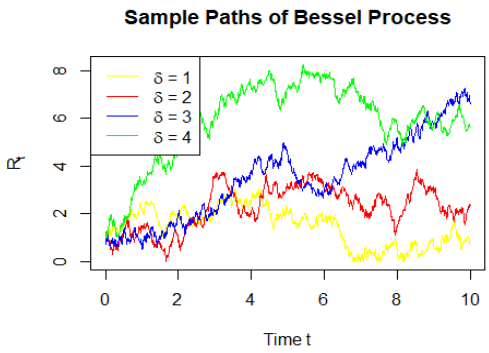
$$dR_t = \frac{\delta - 1}{2R_t}\, dt + dW_t, \quad R_0 = 1$$

Consider the SDE for the squared Bessel process $Y_t = R_t^2$, by Itô's formula:

$$\begin{aligned} dY_t &= 2R_t\, dR_t + \frac{1}{2} \cdot 2\, d[R]_t, \quad \text{where } d[R]_t = dt \\ &= 2R_t \left(\frac{\delta - 1}{2R_t}\, dt + dW_t \right) + dt \\ &= \delta\, dt + 2\sqrt{Y_t}\, dW_t, \quad dW_t \sim \sqrt{dt} \cdot \mathcal{N}(0,1) \\ Y_t &= Y_0 + \int_0^t \delta\, ds + \int_0^t 2\sqrt{Y_s}\, dW_s, \quad Y_0 = R_0^2 = 1 \end{aligned}$$

Check: $\mathbb{E}(Y_t) = Y_0 + \delta t$, since $\mathbb{E}\left(\int_0^t 2\sqrt{Y_s}\, dW_s \right) = 0$

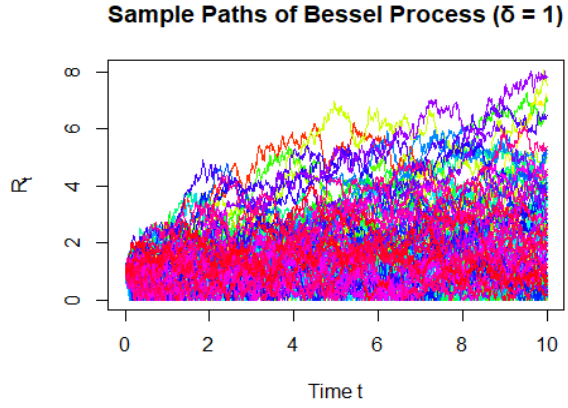
```
set.seed(1)
T = 10
N = 1000
dt = T/N
Rt_sim <- function(delta, N, dt){
  Y <- numeric(N+1)
  Y[1] <- 1
  for (i in 1:N){
    dW <- sqrt(dt)*rnorm(1)
    Y[i+1] <- max(Y[i] + delta*dt+ 2*sqrt(Y[i])*dW, 0)
  }
  return(sqrt(Y))
}
t <- seq(0, T, by = dt)
delta <- c(1,2,3,4)
Rt1 <- Rt_sim(delta[1], N=1000, dt=10/1000)
Rt2 <- Rt_sim(delta[2], N=1000, dt=10/1000)
Rt3 <- Rt_sim(delta[3], N=1000, dt=10/1000)
Rt4 <- Rt_sim(delta[4], N=1000, dt=10/1000)
ymin <- min(c(Rt1, Rt2, Rt3, Rt4))
ymax <- max(c(Rt1, Rt2, Rt3, Rt4))
plot(t, Rt1, "l", col = "yellow", ylim = c(ymin, ymax), main = "Sample Paths of Bessel
Process", xlab = "Time t", ylab = expression(R[t]))
lines(t, Rt2, "l", col="red")
lines(t, Rt3, "l", col="blue")
lines(t, Rt4, "l", col="green")
legend("topleft",
       legend = c(expression(delta *' = 1'), expression(delta *' = 2'),
expression(delta *' = 3'), expression(delta *' = 4')),
       col = c("yellow", "red", "blue", "green"),
       lty = 1)
```



(b)

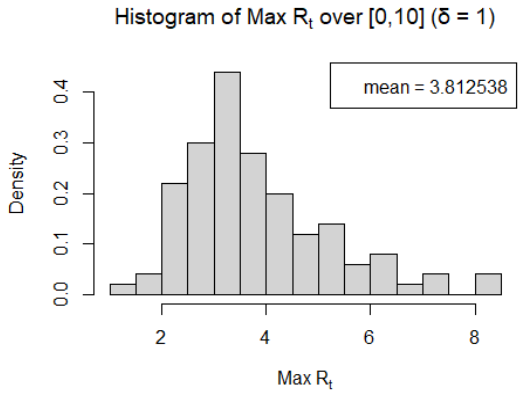
Generate more paths over $t \in [0,10]$ with $\delta = 1$:

```
set.seed(1)
M <- 100  # Number of sample paths
t <- seq(0, T, by = dt)
# choose delta = 1, generate more paths
Rt_matrix <- sapply(1:M, function(m) Rt_sim(delta[1], N, dt)) # row: dt, col: Rt
matplot(t, Rt_matrix, type = "l", lty = 1, col = rainbow(M),
        main = "Sample Paths of Bessel Process (δ = 1)",
        xlab = "Time t", ylab = expression(R[t]))
```



Simulate the maximum for each path and calculate the mean of this maximum:

```
max_Rt <- apply(Rt_matrix, 2, max) # calculate maximum for each column of Rt_matrix
hist(max_Rt, breaks = 20, prob = T,
     main = expression("Histogram of Max " * R[t] * " over [0,10] (δ = 1)"),
     xlab = expression("Max " * R[t]))
mean(max_Rt)

## [1] 3.812538

legend("topright", legend = "mean = 3.812538")
```



(c)

Consider a Bermudan call option with the following characteristics:

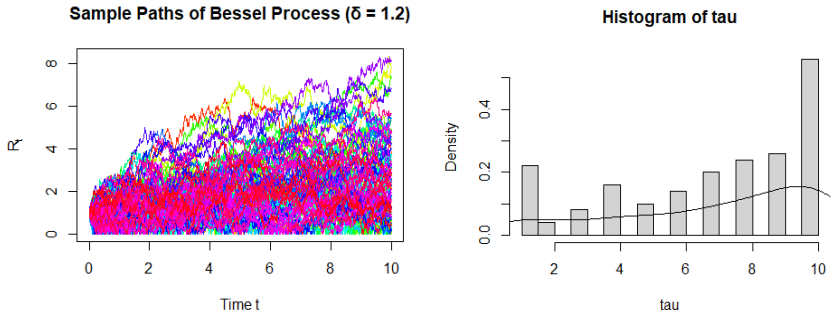
- Underlying stock price: R_t, a Bessel process with order $\delta = 1.2$
- Strike price: $K = 2$
- Maturity: $T = 10$
- Exercisable at discrete times: $t_k = k$, for $k = 1,2,\dots,10$
- Exercise time: τ is a stopping time chosen from $\{t_1, t_2, \dots, t_{10}\}$
- Payoff at exercise: $(R_\tau - K)^+ = \max(R_\tau - K, 0)$

First, generate sample paths for stock price R_t with $\delta = 1.2$.

```
set.seed(1)
delta = 1.2
K = 2
T = 10
M <- 100  # Number of sample paths
t <- seq(0, T, by = dt)
Rt_matrix <- sapply(1:M, function(m) Rt_sim(delta, N, dt)) # rows: t, cols: the mth
path of Rt
matplot(t, Rt_matrix, type = "l", lty = 1, col = rainbow(M),
        main = "Sample Paths of Bessel Process (δ = 1.2)",
        xlab = "Time t", ylab = expression(R[t]))
```

Second, construct a payoff matrix for exercisable dates. The option will be exercised when the highest payoff is reached. Assign these values to the exercise time τ.

```
tk_index <- (1:10)/(10/1000) + 1
Rtk <- Rt_matrix[tk_index, ]  # rows: tk, cols: the mth path of Rt
tk_payoff <- pmax(Rtk - K, 0)  # (Rt - K)+
exercise_time <- apply(tk_payoff, 2, function(payoffs) {
  idx <- which.max(payoffs)  # highest positive payoff
  if (is.na(idx)) return(NA)   # never exercised
  return(idx)  # tk index 1:10
})
# Convert index to actual time τ
tau <- ifelse(is.na(exercise_time), NA, exercise_time)  # τ in 1:10
hist(tau, prob = T, breaks = 20)
lines(density(tau[!is.na(tau)]))
```




```
summary(tau) # median = 8, mean = 6.89
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    4.75    8.00    6.89   10.00   10.00
```

In conclusion, the result (median=8, mean=6.89) shows that there are no incentives to exercise early. Since no discounting factors exist, most options are exercised close to maturity.

(d)

Assuming no discounting factor, i.e. $r = 0$ thus $e^{-rt} = 1$, the pricing formula of the Bermuda call option is $E\left(max\left(\left(R_{t_i} - K\right)^+, E(V_{i+1}|Fi)\right)\right)$, where V_{i+1} is the future value. Hence this is equivalent to find the maximum value of the payoffs all through the exercisable times, i.e. $max\left(\left(R_{t_1} - K\right)^+, \ldots, \left(R_{t_{10}} - K\right)^+\right)$

```
Bermuda_payoffs <- sapply(1:M, function(m) {
  payoffs <- tk_payoff[, m]
  holding_values <- numeric(10)
  holding_values[10] <- payoffs[10]
  for (t in 9:1) {
    if (payoffs[t] > 0) {
      immediate_exercise <- payoffs[t]
      future_payoff <- holding_values[t + 1]
      holding_values[t] <- max(immediate_exercise, future_payoff)
    } else {
      holding_values[t] <- holding_values[t + 1]
    }
  }
  return(holding_values[1])
})
Bermuda_price <- mean(Bermuda_payoffs)
cat("Bermuda call price =", Bermuda_price, "\n")
```

```
## Bermuda call price = 1.681711
```

The European call option price $= E((R_{10} - K)^+)$.

```
# European call option price = E((R10-K)+)
European_payoff <- tk_payoff[10,]
European_price <- mean(European_payoff)
cat("European call price =", European_price, "\n")
```

```
## European call price = 1.050808
```

Calculate how much more expensive the Bermuda call option is than a European call option:

```
diff_price = Bermuda_price - European_price
cat("Extra cost of Bermuda over European =", diff_price, "\n")
```

```
## Extra cost of Bermuda over European = 0.630903
```

(e)

(1) No proper modeling of expected future values:

We used a single "average of future payoffs" or just "the next time step's payoff," which might be a coarse approximation. The more accurate method in continuous models is to do a regression across all paths' next-step (or multiple-step) payoffs on the underlying Rt.

(2) Assumption of the discounting factor is not realistic:

In real-world financial markets, option pricing typically incorporates a positive risk-free interest rate $r > 0$, leading to exponential discounts of future payoffs.

When discounting is present, the option value at an exercise time t_i is given by:

$$V_i = \max\left(\left(R_{t_i} - K\right)^+,\ e^{-r\Delta t} \cdot \mathbb{E}\left[V_{i+1} \mid \mathcal{F}_{t_i}\right]\right)$$

The second term represents the discounted expected continuation value, where Δt is the time between exercise opportunities, and $\mathcal{F}_{t_i}$ denotes the available information at time t_i.

In contrast to a zero-interest-rate setting where the continuation value is not discounted, a positive r reduces the present value of waiting. This provides a stronger incentive for early exercise and typically increases the premium of the Bermuda option over its European counterpart.

(3) Limited number of paths:

If the number of paths is relatively small (e.g. we use M=100 in this case), the variance of the estimator might be high. Checking for confidence intervals or repeating with a bigger M can highlight whether results have converged.

(4) Issues arising from the direct simulation of Rt:

$$R_t \to 0 \quad \Rightarrow \quad Y_t = R_t^2 \to 0$$

To avoid taking the square root of negative values when simulating $R_t = \sqrt{Y_t}$, we applied a non-negativity constraint in the simulation of Y_t:

$$Y_{i+1} = \max\left(Y_i + \delta \cdot \Delta t + 2\sqrt{Y_i} \cdot \Delta W_i,\ 0\right)$$

This ensures that $Y_t \geq 0$, so that $R_t = \sqrt{Y_t}$ is always real-valued.

(5) Variance Reduction:

Control Variates Method:

The price of the Bermuda call option is equal to $max\left(\left(R_{t_1} - K\right)^+, \ldots, \left(R_{t_{10}} - K\right)^+\right)$, which is equivalent to $max(max(R_t) - K,\ 0)$.

Let $X = max(max(R_t) - K,\ 0),\ for\ t = 1,2,\ldots,10$. Set $\tilde{X} = max(R_t) - K$, where we can use similar method in (b) to calculate $max(R_t)$, then $Y = \tilde{X} - E(\tilde{X})$. Check: $E(Y) = 0$.

```
max_Rtk <- apply(Rtk, 2, max)
X_tilde = max_Rtk - K
X = pmax(X_tilde, 0)
Y = X_tilde - mean(X_tilde)
mean(Y) # check E(Y)=0

## [1] -7.773459e-17
```

Calculate $c = -\frac{Cov(X,Y)}{Var(Y)}$:

```
c <- -cov(X,Y)/var(Y)
c

## [1] -0.9687942
```

Then $\hat{\theta} = X + cY$, calculate $Var(\hat{\theta})$ and compare it to $Var(X)$:

```
theta_hat = X + c*Y
cat(var(X), var(theta_hat), "\n",
    "Improved by",round((var(X)-var(theta_hat))/var(X) * 100, digits = 2), "%.")

## 2.030454 0.01265765
##  Improved by 99.38 %.
```

Other attempts for variance reduction:

1. Antithetic Variates Method: unavailable since R_t is not a monotone function.
2. Stratify Sampling: unavailable since an inverse transformed estimator is needed.
3. Conditioning Sampling: unavailable since R_t is the only variable.
4. Importance Sampling: unavailable since the distribution of R_t is not given.

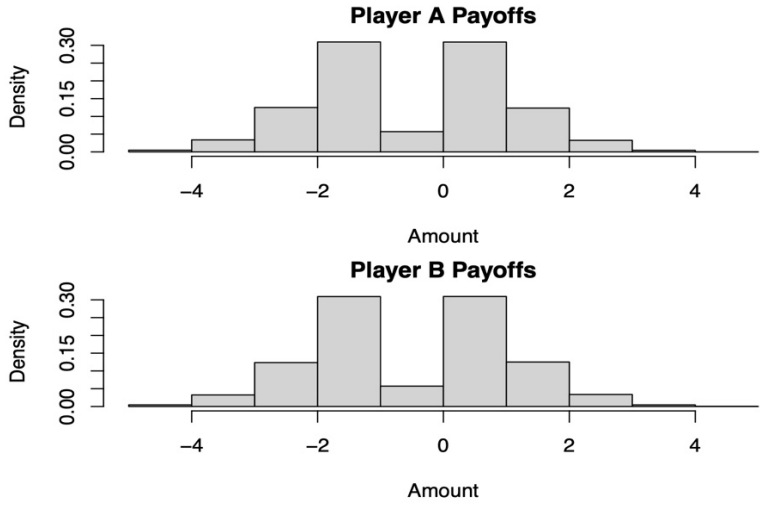
Question 4

(a)

First, we create a function that models one game and builds a deck with numbers 1-8 (with four numbers repeated).

Then, we randomly deal 4 cards to each player. The player who has the lowest sum will be the winner of the game.

```
sim_game <- function() {
  nums <- 1:8
  repeat_nums <- sample(nums, size=4, replace = FALSE)
  my_deck <- c(nums, repeat_nums)

  # Draw cards without replacement
  cards_drawn <- sample(my_deck, size=8, replace = FALSE)

  # Split cards between players
  playerA_cards <- cards_drawn[1:4]
  playerB_cards <- cards_drawn[5:8]

  # Find totals
  pA_total <- sum(playerA_cards)
  pB_total <- sum(playerB_cards)

  # Figure out who wins and calculate the payoff
  result <- sign(pB_total - pA_total)

  if (result > 0) {  # Player A wins
    pay <- min(playerA_cards)
    return(c(pay, -pay))
  } else if (result < 0) {  # Player B wins
    pay <- min(playerB_cards)
    return(c(-pay, pay))
  } else {  # Nobody wins
    return(c(0, 0))}}

# Run simulation 100,000 times
set.seed(42)
sims <- 100000
all_results <- matrix(0, nrow = sims, ncol = 2)
for (i in 1:sims) all_results[i,] <- sim_game()

# Find unique outcomes for histograms
pA_outcomes <- sort(unique(all_results[,1]))
pB_outcomes <- sort(unique(all_results[,2]))

# Make histograms
par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))
hist(all_results[,1], breaks = pA_outcomes, prob = TRUE,
     main = "Player A Payoffs", xlab = "Amount")
hist(all_results[,2], breaks = pB_outcomes, prob = TRUE,
     main = "Player B Payoffs", xlab = "Amount")
```




```
# Calculate expected values
avg_pA <- mean(all_results[,1])
avg_pB <- mean(all_results[,2])
cat("\nPlayer A expected value:", round(avg_pA, 4))
## Player A expected value: -0.0077

cat("\nPlayer B expected value:", round(avg_pB, 4))
## Player B expected value: 0.0077
```

Conclusion:
The payoff distributions for both players are mirror images of each other.
The most common outcomes are -1, 0, 1, and 2, with less frequent outcomes at -3, -2, 3, and 4.
The expected values (Player A: -0.0077, Player B: 0.0077) are essentially zero.
The game appears to be fair, with neither player having a significant advantage.
Ties (payoff of 0) occur frequently, as shown by the peak at 0 in both histograms.

(b)

```
# Create a new function where Player A always gets their chosen card
sim_game_modified <- function(chosen) {
  nums <- 1:8
  repeat_nums <- sample(nums, size=4, replace = FALSE)
  my_deck <- c(nums, repeat_nums)

  # Player A gets a chosen card plus 3 random cards
  # Player B gets 4 random cards
  if (chosen %in% my_deck) {
    idx <- which(my_deck == chosen)[1]
    my_deck <- my_deck[-idx]
  } else {
    return(sim_game_modified(chosen)) }

  cards_drawn <- sample(my_deck, size=7, replace = FALSE)
  playerA_cards <- c(chosen, cards_drawn[1:3])
  playerB_cards <- cards_drawn[4:7]

  pA_total <- sum(playerA_cards)
  pB_total <- sum(playerB_cards)

  if (pA_total < pB_total) {
    pay <- min(playerA_cards)
    return(c(pay, -pay))
  } else if (pB_total < pA_total) {
```

```
    pay <- min(playerB_cards)
    return(c(-pay, pay))
  } else {
    return(c(0, 0))
  }}

# Test each possible card choice (1-8) with 50,000 simulations
set.seed(42)
sims <- 50000
card_choices <- 1:8
choice_results <- matrix(0, nrow = 8, ncol = 2)
colnames(choice_results) <- c("Exp Value", "Win %")

for (i in 1:8) {
  test_results <- matrix(0, nrow = sims, ncol = 2)
  for (j in 1:sims) test_results[j,] <- sim_game_modified(i)

  choice_results[i,] <- c(
    mean(test_results[,1]),
    mean(test_results[,1] > 0) * 100
  )}

# Print results
cat("\nCard choice results (Expected Value, Win %):")


## Card choice results (Expected Value, Win %):

for (i in 1:8) {
  cat("\nCard", i, ":", round(choice_results[i,1], 4),
      ", ", round(choice_results[i,2], 1), "%")}


## Card 1 : 0.294 ,  70.9 %
## Card 2 : 0.518 ,  63.4 %
## Card 3 : 0.4087 ,  56.6 %
## Card 4 : 0.2018 ,  50 %
## Card 5 : -0.0165 ,  43.9 %
## Card 6 : -0.2471 ,  37.4 %
## Card 7 : -0.4956 ,  30.5 %
## Card 8 : -0.747 ,  23.5 %

best <- which.max(choice_results[,1])
cat("\n\nBest choice for Player A: Card", best)
## Best choice for Player A: Card 2

cat("\nThis gives expected value of", round(choice_results[best,1], 4), "per game")
## This gives an expected value of 0.518 per game

cat("\nThe fair game had expected value around 0, so this is definitely better")
## The fair game had an expected value of around 0, so this is definitely better
```

Conclusion:
The player A should choose card 2. It gives the highest expected value of payoff equal to 0.518 per game.
While choosing card 1 offers the highest win probability (70.9%), it only yields an expected value of payoff equal to 0.294. Card 2 provides a better balance of win probability (63.4%) and a higher payoff when winning.
As card numbers increase, both expected values of payoff and win probabilities decrease steadily.
Player A now has a significant advantage.